

- HIA Architecture Overview (IPHIPI Hackathon 2026)
 - 1) Problem Summary and User Journey
 - 2) High-Level Architecture
 - Frontend (React + Vite + Tailwind)
 - Backend (FastAPI)
 - Data Store (SQLite)
 - 3) Module/Agent Interaction
 - 4) Key Design Choices and Trade-offs
 - Latency
 - Scalability
 - Model selection
 - Reliability trade-off
 - 5) Scoring and Aggregation
 - 6) Limitations, Assumptions, Next Steps
 - Current limitations
 - Assumptions
 - Next steps

HIA Architecture Overview (IPHIPI Hackathon 2026)

1) Problem Summary and User Journey

Candidates need realistic, adaptive interview practice instead of static question banks. HIA delivers a multimodal mock interview flow that:

1. Uploads and parses resume
2. Recommends role tracks
3. Runs automatic voice interview rounds (no manual record/submit flow)
4. Captures audio + CV signals
5. Produces round-wise and final coaching summary
6. Stores sessions/interviews for review and longitudinal progression

User journey:

Dashboard -> Upload Resume -> Processing -> Role Selection -> Live Interview -> Summary -> Review past sessions/interviews

2) High-Level Architecture

Frontend (React + Vite + Tailwind)

- Screens: Dashboard, Upload, Processing, Roles, Live Interview, Summary
- State: global app context (**AppContext**)
- API client: axios service layer
- Interview UI is event-driven:
 - TTS speaking
 - listening/recording
 - silence timeout
 - transcription/evaluation transition
 - next question / complete

Backend (FastAPI)

- **api/server.py** orchestrates runtime endpoints
- Resume intelligence and job matching modules
- Question retrieval + generation orchestration
- Audio transcription + metric extraction
- CV frame analysis
- Multimodal score fusion
- Session/interview persistence in SQLite

Data Store (SQLite)

- File: **data/processed_data/interview_sessions.db**
- Tables:
 - **resumes**
 - **sessions**
 - **interviews**

- `interview_memory`
 - `resume_files`
-

3) Module/Agent Interaction

1. Resume layer

- `resume_agent.py` extracts structured candidate profile
- `recommend_jobs.py` + embeddings generate top role recommendations
- `resume_registry.py` deduplicates using resume hash

2. Session layer

- `session_manager.py` creates/continues sessions
- max interviews per session enforced
- interview summaries persisted as JSON per interview row

3. Interview orchestration

- `interview_orchestrator.py` plans question-type distribution
- retrieval path: `question_retriever.py`
- generation path: role/resume/skill specific question generators
- intro question is guaranteed first (HIA introduction)

4. Audio pipeline

- Browser captures answer until silence timeout
- backend `/transcribe_audio` runs Whisper + fallback variants
- computes communication-related features (pause/WPM/etc)

5. CV pipeline

- frontend sends sampled webcam frames
- backend `VisionPipeline` returns engagement/eye-contact/attention and related signals

6. Scoring/fusion

- `answer_evaluator.py` technical answer quality

- `multimodal_evaluator.py` merges technical + audio/CV into final round score and coaching
-

4) Key Design Choices and Trade-offs

Latency

- Chosen for live usability over heavy batch analytics.
- CV is sampled periodically (not per-frame streaming) to reduce overhead.

Scalability

- SQLite is ideal for hackathon/local single-node workflows.
- Design isolates session/data managers for future move to Postgres/Mongo.

Model selection

- Whisper (`faster-whisper`) for robust local STT.
- Sentence-transformers for retrieval embedding matching.
- MediaPipe-based CV stack for lightweight local inference.

Reliability trade-off

- If waveform decode fails for browser audio container, backend now safely falls back instead of crashing.
 - This prevents interview dead-ends while preserving transcript continuity.
-

5) Scoring and Aggregation

Per round:

1. Technical score from LLM answer evaluator.

2. Audio signals (confidence, pauses, pace, fillers, consistency).
3. CV signals (engagement, eye contact, attention; warnings).
4. Fusion logic outputs:
 - technical score
 - communication score
 - confidence score
 - overall score
 - coaching feedback (technical/communication/behavioral)
 - optional warnings in coaching mode

Final summary:

- Aggregates all rounds with category/difficulty history and full Q/A/evaluation details.
 - Stored in DB as `interview_summary_json`, reviewable later from dashboard.
-

6) Limitations, Assumptions, Next Steps

Current limitations

- Browser TTS behavior varies by browser/OS speech engine.
- CV quality depends on lighting/camera angle.
- SQLite is not ideal for high-concurrency production traffic.

Assumptions

- Candidate grants mic/camera permissions.
- Local machine has sufficient CPU for Whisper + CV.
- OpenRouter key is configured and has credits.

Next steps

- WebSocket-based real-time status/events instead of polling style transitions.
- Stronger anti-repetition constraints across long session history.
- More explicit rubric scoring (structure, STAR usage, evidence quality).
- Production migration path (Postgres + background workers + observability).